

simple_cp.c

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {
    FILE *source_file, *dest_file;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    /* Check if the user provided the correct number of command-line arguments */
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source-file> <dest-file>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    /* Open the source file for reading in binary mode */
    source_file = fopen(argv[1], "rb");
    if (source_file == NULL) {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }

    /* Open the destination file for writing in binary mode */
    dest_file = fopen(argv[2], "wb");
    if (dest_file == NULL) {
        perror("Error opening destination file");
        fclose(source_file);
        exit(EXIT_FAILURE);
    }

    /* Read from source and write to destination in chunks */
    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, source_file)) > 0) {
        if (fwrite(buffer, 1, bytes_read, dest_file) != bytes_read) {
            perror("Error writing to destination file");
            fclose(source_file);
            fclose(dest_file);
            exit(EXIT_FAILURE);
        }
    }

    /* Close the file streams */
    fclose(source_file);
    fclose(dest_file);

    return EXIT_SUCCESS;
}
```

simple_cp_syscall.c

```
#include <fcntl.h> /* Contains system calls for file control (open) */
#include <unistd.h> /* Contains system calls for POSIX API (read, write, close) */
#include <stdio.h> /* Used here only for perror() and printing */
#include <stdlib.h>
```

```
#define BUFFER_SIZE 4096
```

```
int main(int argc, char *argv[]) {
    int src_fd, dest_fd; /* File descriptors: integers representing open files */
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    /* Verify correct command-line arguments */
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source-file> <dest-file>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    /*
     * open() is a system call. It tells the Kernel: "I want to access this file."
     * The Kernel checks permissions and returns a 'File Descriptor' (an integer).
     * O_RDONLY: Open the file for reading only.
     */
    src_fd = open(argv[1], O_RDONLY);
    if (src_fd == -1) {
        perror("Failed to open source file");
        exit(EXIT_FAILURE);
    }

    /*
     * O_WRONLY: Open for writing.
     * O_CREAT: Create the file if it doesn't exist.
     * O_TRUNC: If the file exists, wipe its content (truncate to 0 length).
     * 0644: Permissions (owner can read/write, others can read).
     */
    dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd == -1) {
        perror("Failed to create destination file");
        close(src_fd);
        exit(EXIT_FAILURE);
    }

    /*
     * read() is a system call. It requests the Kernel to copy data from the
     * file into our buffer. The Kernel puts the process to sleep until the
     * disk/filesystem delivers the data.
     */
}
```

```

while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
    /*
     * write() is a system call. It asks the Kernel to take data from our
     * buffer and place it into the destination file.
     */
    bytes_written = write(dest_fd, buffer, bytes_read);
    if (bytes_written != bytes_read) {
        perror("Error writing data");
        close(src_fd);
        close(dest_fd);
        exit(EXIT_FAILURE);
    }
}

/*
 * close() is a system call. It informs the Kernel that we are done.
 * The Kernel can now free the resources associated with these file descriptors.
 */
close(src_fd);
close(dest_fd);

return EXIT_SUCCESS;
}

```

simple_cp_windows.c

```
#include <windows.h>
```

```
#include <stdio.h>
```

```
#define BUFFER_SIZE 4096
```

```
int main(int argc, char *argv[]) {  
    HANDLE hSource, hDest;  
    char buffer[BUFFER_SIZE];  
    DWORD bytesRead, bytesWritten;
```

```
    if (argc != 3) {  
        printf("Usage: %s <source-file> <dest-file>\n", argv[0]);  
        return 1;  
    }
```

```
    /*  
    * CreateFile is the Windows equivalent of open().  
    * It returns a HANDLE, which is an opaque pointer/index  
    * managed by the Windows Object Manager.  
    */
```

```
    hSource = CreateFileA(argv[1], GENERIC_READ, FILE_SHARE_READ, NULL,  
        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
```

```
    if (hSource == INVALID_HANDLE_VALUE) {  
        printf("Could not open source file. Error: %lu\n", GetLastError());  
        return 1;  
    }
```

```
    /* Create/Truncate destination file */
```

```
    hDest = CreateFileA(argv[2], GENERIC_WRITE, 0, NULL,  
        CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
```

```
    if (hDest == INVALID_HANDLE_VALUE) {  
        printf("Could not create destination file. Error: %lu\n", GetLastError());  
        CloseHandle(hSource);  
        return 1;  
    }
```

```
    /* Read and Write loop using Windows API */
```

```
    while (ReadFile(hSource, buffer, BUFFER_SIZE, &bytesRead, NULL) &&  
        bytesRead > 0) {  
        if (!WriteFile(hDest, buffer, bytesRead, &bytesWritten, NULL) || bytesWritten !=  
            bytesRead) {  
            printf("Error writing to file. Error: %lu\n", GetLastError());  
            break;  
        }  
    }
```

```
/* Close handles to release kernel resources */  
CloseHandle(hSource);  
CloseHandle(hDest);  
  
return 0;  
}
```